THAO TÁC BIT & MEET IN THE MIDDLE

Chuyên đề: Kỹ thuật Xử lý Bit và Tối ưu hóa Duyệt Tập hợp

Tài liệu Giảng dạy Lập trình Thi đấu (Olympiad, ICPC)

Thuật toán & Cấu trúc Dữ liệu

October 26, 2026

Cấu trúc nội dung bài học

Phần I: Nền tảng Thao tác Bit

1. **Dãy bit & Biểu diễn số trong máy tính**
   - Hệ nhị phân (cơ số 2) và trọng số 2^i.
   - Giới hạn bit của các kiểu dữ liệu nguyên thủy.
2. **Các phép toán Bitwise cơ bản**
   - Phép toán logic: AND (&), OR (|), XOR (^).
   - Phép dịch bit: Dịch trái (<<) và Dịch phải (>>).
3. **Các kỹ thuật xử lý từng Bit**
   - Kiểm tra trạng thái bit thứ i.
   - Bật (Set), Tắt (Clear) và Đảo (Flip) bit thứ i.

Phần II: Kỹ thuật Duyệt & Tối ưu hóa

4. **Duyệt phân tập (Subset Generation)**
   - Biểu diễn tập con bằng mặt nạ bit (Bitmask).
   - Cài đặt chuẩn và giới hạn ứng dụng ($n \leq 20$).
5. **Kỹ thuật chia đôi tập (Meet in the Middle)**
   - Bài toán Subset Sum với giới hạn $n \leq 40$.
   - Tư tưởng chia đôi không gian trạng thái.
   - Kết hợp hai nửa bằng Tìm kiếm nhị phân.
   - Phân tích độ phức tạp $\mathcal{O}(n \cdot 2^{n/2})$.

Nguyên lý biểu diễn nhị phân

- Máy tính không lưu trữ trực tiếp hệ thập phân (cơ số 10) như con người sử dụng hằng ngày.
- Mọi dữ liệu số và ký tự đều được mã hóa về **hệ nhị phân (cơ số 2)**, chỉ gồm hai trạng thái điện học là bit 0 và bit 1.
- Các bit được đánh chỉ số (index) từ **phải sang trái**, bắt đầu từ vị trí 0 (bit có trọng số nhỏ nhất – LSB).

Giới hạn bit của kiểu dữ liệu (C++)

- Mỗi kiểu dữ liệu nguyên thủy sử dụng một số lượng bit cố định trong bộ nhớ:
  - Kiểu int: gồm 32-bit (1 bit dấu, 31 bit giá trị, miền giá trị xấp xỉ $[-2 \cdot 10^9, 2 \cdot 10^9]$).
  - Kiểu long long: gồm 64-bit (1 bit dấu, 63 bit giá trị, miền giá trị xấp xỉ $[-9 \cdot 10^{18}, 9 \cdot 10^{18}]$).
- Mọi thao tác vượt quá số bit quy định sẽ gây ra lỗi **tràn số (overflow)**.

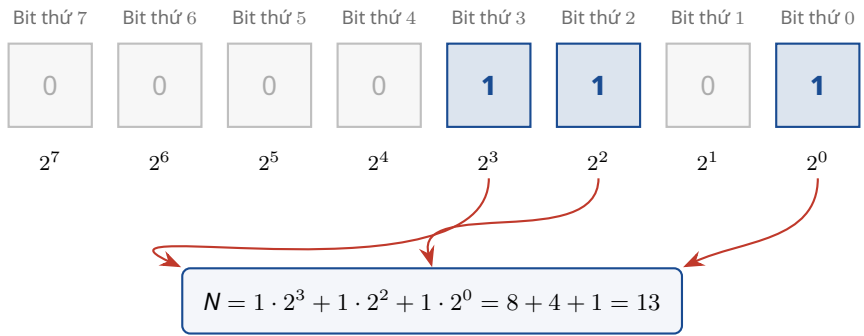
Công thức tổng quát chuyển đổi Nhị phân sang Thập phân

- Một số nguyên không âm N biểu diễn bởi dãy bit $b_k b_{k-1} \dots b_1 b_0$ có giá trị thập phân được tính bằng tổng các lũy thừa của 2:

$$N = \sum_{i=0}^{k} b_i \cdot 2^i = b_k \cdot 2^k + b_{k-1} \cdot 2^{k-1} + \cdots + b_1 \cdot 2^1 + b_0 \cdot 2^0 \quad \text{với } b_i \in \{0, 1\}$$

- **Nhận xét:** Chỉ những vị trí có bit bật ($b_i = 1$) mới đóng góp giá trị 2^i vào tổng thập phân của số N.

Visualization: Biểu diễn số $N = 13$ dưới dạng dãy 8-bit (00001101)



- Các bit ở vị trí $i \in \{3, 2, 0\}$ mang giá trị 1, tương ứng với các trọng số $2^3 = 8$, $2^2 = 4$, và $2^0 = 1$.
- Các bit ở vị trí $i \in \{7, 6, 5, 4, 1\}$ mang giá trị 0 nên không đóng góp vào giá trị của N.

2.1. Các phép toán Logic trên Bit: AND (&), OR (|), XOR (^)

1. Phép toán AND (&)

- **Quy tắc:** Kết quả bằng 1 nếu và chỉ nếu **cả hai** bit cùng vị trí đều là 1; ngược lại bằng 0.
- **Đặc tính:** $x \,\&\, 0 = 0$ và $x \,\&\, 1 = x$.
- **Ứng dụng:** Dùng để **tắt (clear)** các bit về 0 hoặc **trích xuất (lọc)** giá trị của một nhóm bit nhất định.

2. Phép toán OR (|)

- **Quy tắc:** Kết quả bằng 1 nếu **ít nhất một trong hai** bit cùng vị trí là 1; chỉ bằng 0 khi cả hai cùng là 0.
- **Đặc tính:** $x \mid 1 = 1$ và $x \mid 0 = x$.
- **Ứng dụng:** Dùng để **bật (set)** ép một hoặc nhiều vị trí bit lên giá trị 1 hoặc hợp nhất hai tập trạng thái.

3. Phép toán XOR (^ hoặc ký hiệu $\oplus$)

- **Quy tắc:** Kết quả bằng 1 nếu hai bit **khác nhau** (0 với 1), và bằng 0 nếu hai bit **giống nhau**.
- **Bản chất:** Là phép cộng không nhớ trong hệ cơ số 2.

- **Tính chất:**
  - Triệt tiêu phần tử giống nhau: $A \oplus A = 0$.
  - Phần tử trung hòa: $A \oplus 0 = A$.
  - Giao hoán & Kết hợp: $A \oplus (B \oplus C) = (A \oplus B) \oplus C$.

Minh họa Đặt tính Bitwise trên từng cột bit

Các phép toán Bitwise thực hiện độc lập trên từng cặp bit cùng cột (không có số nhớ sang cột bên cạnh) với độ phức tạp $\mathcal{O}(1)$.

Phép AND (A & B)

A	1	0	1	0	1	1
& B	0	1	1	0	1	0
	0	0	**1**	0	**1**	0

- Cột tô xanh: cả *A* và *B* đều bằng 1 $\rightarrow$ kết quả bằng 1.

Phép OR (A | B)

A	1	0	1	0	1	1
| B	0	1	1	0	1	0
	1	**1**	**1**	0	**1**	**1**

- Chỉ cột thứ 4 có cả hai bit bằng 0 $\rightarrow$ kết quả bằng 0.

Phép XOR (A ^ B)

A	1	0	1	0	1	1
^ B	0	1	1	0	1	0
	1	**1**	0	0	0	**1**

- Cột tô cam: hai bit đối nghịch nhau $\rightarrow$ kết quả bằng 1.

Bảng chân trị của các phép toán Bitwise

Bit A	Bit B	AND (A & B)	OR (A | B)	XOR ($A \oplus B$)	Trạng thái đầu vào
0	0	0	0	0	Cả hai bit đều tắt
0	1	0	**1**	**1**	Hai bit khác trạng thái
1	0	0	**1**	**1**	Hai bit khác trạng thái
1	1	**1**	**1**	0	Cả hai bit đều bật

Tóm tắt phản xạ nhanh khi làm bài

- **Cần kiểm tra hoặc xóa bit:** Nghĩ ngay đến phép **AND (&)** vì AND với 0 luôn ép bit về 0.
- **Cần gán bit lên 1:** Nghĩ ngay đến phép **OR (|)** vì OR với 1 luôn ép bit lên 1.
- **Cần lật trạng thái bit hoặc tìm phần tử xuất hiện lẻ lần:** Nghĩ ngay đến phép **XOR (^)** vì $x \oplus 1 = \neg x$ và $x \oplus x = 0$.

2.2. Phép Dịch trái (<<) và Dịch phải (>>)

1. Phép Dịch trái – Shift Left ($A \ll k$)

- **Cơ chế hoạt động:**
  - Đẩy toàn bộ dãy bit của số A sang bên trái k vị trí.
  - k vị trí trống mới sinh ra ở phía bên phải sẽ được điền đầy bằng các bit 0.
- **Ý nghĩa toán học:**

$$A \ll k = A \cdot 2^k$$

- **Trường hợp đặc biệt rất hay dùng:**

$$1 \ll k = 2^k$$

- **Lỗi tràn số:** Biểu thức 1 << k mặc định tính trên kiểu int (32-bit). Nếu $k \geq 31$, bắt buộc phải viết 1LL << k để ép sang kiểu long long (64-bit).

2. Phép Dịch phải – Shift Right ($A \gg k$)

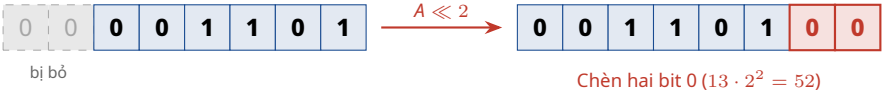
- **Cơ chế hoạt động:**
  - Đẩy toàn bộ dãy bit của số A sang bên phải k vị trí.
  - k bit ngoài cùng bên phải bị đẩy ra khỏi phạm vi biểu diễn sẽ bị **loại bỏ hoàn toàn** (rơi mất bit).
- **Ý nghĩa toán học:**

$$A \gg k = \left\lfloor \frac{A}{2^k} \right\rfloor$$

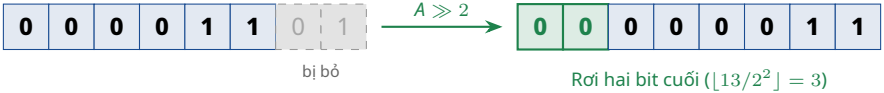
- **Ứng dụng phổ biến:**
  - Chia đôi nhanh trong Tìm kiếm nhị phân hoặc Segment Tree: mid = (l + r) >> 1.
  - Đưa bit ở vị trí thứ i về vị trí cuối cùng (index 0) để kiểm tra giá trị của nó.

Minh họa Phép Dịch trái (<<) và Dịch phải (>>)

1. Minh họa Dịch trái $A \ll 2$ với $A = 13$ (00001101):



2. Minh họa Dịch phải $A \gg 2$ với $A = 13$ (00001101):



3.1. Kiểm tra trạng thái của bit thứ *i*

Tư tưởng Mặt nạ Bit (Bitmask) & Công thức kiểm tra

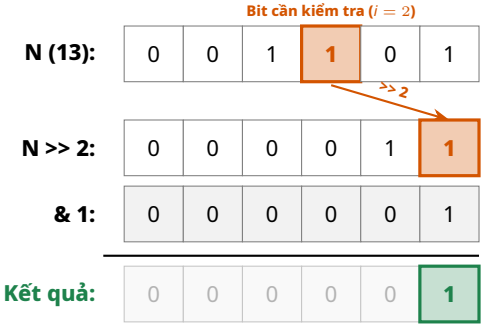
- Trong lập trình thi đấu, ta thường dùng một số nguyên duy nhất để thay thế cho cả một mảng đánh dấu bool (gồm các trạng thái True/False tương ứng với bit 1/0).
- Để trích xuất giá trị của bit ở vị trí thứ *i* (tính từ phải sang trái, bắt đầu từ 0) của số nguyên *N*, ta sử dụng biểu thức:

```
int bit_i = (N >> i) & 1;
```

Giải thích cơ chế hoạt động qua 3 bước

1. **Bước 1 – Dịch bit cần tìm về cuối:** Phép toán N >> i dịch toàn bộ dãy bit của *N* sang phải *i* vị trí, đưa bit thứ *i* về đúng vị trí tận cùng bên phải (index 0).
2. **Bước 2 – Áp mặt nạ cô lập:** Thực hiện phép AND với số 1 (có dạng nhị phân 000...0001).
3. **Bước 3 – Thu nhận kết quả:** Do mọi vị trí từ index 1 trở đi của số 1 đều là bit 0, phép & 1 sẽ xóa sạch toàn bộ các bit phía trước về 0 và chỉ giữ lại nguyên vẹn giá trị của bit ở vị trí index 0.

Kiểm tra trạng thái bit $i = 2$ của $N = 13$ (001101)



Quy trình

- Ban đầu, bit thứ 2 của $N = 13$ nằm ở ô màu cam (giá trị 1).
- Sau phép dịch N >> 2, ô màu cam được dời về vị trí index 0 (ngoài cùng bên phải).
- Khi thực hiện & 1, bit 1 ở index 1 bị triệt tiêu thành 0, chỉ còn lại ô màu xanh lá mang giá trị 1.
- **Kết luận:** Bit thứ 2 của $N = 13$ đang ở trạng thái **Bật (1)**.

3.2. Bật (Set), Tắt (Clear) và Đảo (Flip) bit ở vị trí thứ *i*

1. Bật (Set) bit thứ *i* thành 1

- **Công thức:**

```
N = N | (1 << i);    hay    N |= (1 << i);
```

- **Giải thích:**
  - Biểu thức (1 << i) tạo ra một mặt nạ (mask) chỉ có duy nhất bit thứ *i* là 1, tất cả các vị trí khác đều là 0.
  - Khi lấy N thực hiện phép OR (|) với mặt nạ này, bit thứ *i* chắc chắn biến thành 1, còn các bit khác của N giữ nguyên giá trị cũ (do $x \mid 0 = x$).

2. Tắt (Clear) và Đảo (Flip) bit thứ *i*

- **Đảo trạng thái bit thứ *i* (0 $\leftrightarrow$ 1):**

```
N ^= (1 << i);
```

  - Dựa vào tính chất $x \oplus 1 = \neg x$ và $x \oplus 0 = x$.

- **Tắt bit thứ *i* về 0:**

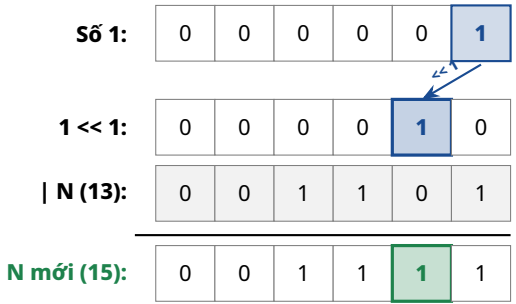
```
N &= ~(1 << i);
```

  - Toán tử ~ đảo ngược mặt nạ (chỉ bit *i* bằng 0, còn lại bằng 1), sau đó dùng AND để tắt bit *i*.

Lưu ý khi thao tác với số nguyên lớn (long long)

- Khi chỉ số bit $i \geq 31$, tuyệt đối không viết (1 << i) vì hằng số 1 mặc định là kiểu int 32-bit sẽ bị tràn số.
- **Khắc phục:** Luôn viết **(1LL << i)** khi làm việc với kiểu long long.

Bật bit thứ $i = 1$ của $N = 13$ (001101)



Quy trình

- Ở số $N = 13$ (001101), bit ở vị trí $i = 1$ đang mang giá trị 0.
- Phép dịch trái 1 << 1 tạo ra mặt nạ 000010 (chỉ bật ô ở vị trí $i = 1$).
- Khi thực hiện phép OR giữa mặt nạ và N, bit ở vị trí $i = 1$ được bật lên 1 (ô màu xanh lá).
- Giá trị mới thu được là $N = 15$ (001111), tức là $13 + 2^1 = 15$.

4. Kỹ thuật ứng dụng: Duyệt phân tập (Subset Generation)

Biểu diễn tập con bằng Bitmask

- Cho tập hợp $S = \{A_0, A_1, \ldots, A_{n-1}\}$ gồm n phần tử.
- Mỗi tập con của S tương ứng song ánh với một dãy n bit:
  - **Bit thứ** $i = 1$**:** Phần tử A_i **có mặt** trong tập con.
  - **Bit thứ** $i = 0$**:** Phần tử A_i **không có mặt**.
- Vì có 2^n tập con, ta chỉ cần cho biến nguyên mask chạy từ 0 đến $2^n - 1$ để vét cạn toàn bộ tập con mà không cần dùng đệ quy quay lui.

Đánh giá Độ phức tạp

- **Thời gian:** $\mathcal{O}(n \cdot 2^n)$.
- **Giới hạn áp dụng:** Chạy tốt trong 1 giây khi $n \leq 20$.

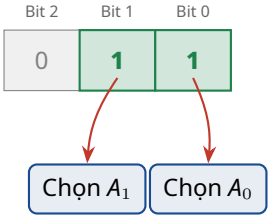
Cài đặt chuẩn bằng C++

```cpp
int n = 3;
// Duyet mask tu 0 den 2^n - 1
for (int mask = 0; mask < (1 << n); ++mask) {
    vector<int> subset;
    for (int i = 0; i < n; ++i) {
        // Kiem tra bit thu i co bat hay khong
        if ((mask >> i) & 1) {
            subset.push_back(i);
        }
    }
    // Xu ly tap con (subset) tai day...
}
```

- **Ưu điểm so với Đệ quy (Backtracking):** Code cực kỳ ngắn gọn, không tốn bộ nhớ ngăn xếp (Call Stack) và tốc độ thực thi thực tế nhanh hơn đáng kể.

Ánh xạ Bitmask sang Tập con với $n = 3$ ($S = \{A_0, A_1, A_2\}$)

Cơ chế ánh xạ tại mask $= 3$ (011)



- Với mask $= 3$, biểu diễn nhị phân là 011.
- Hai bit bật ở vị trí 0 và 1 tương ứng với tập con $\{A_0, A_1\}$.

Bảng liệt kê toàn bộ $2^3 = 8$ trạng thái

Giá trị mask	Nhị phân ($b_2 b_1 b_0$)	Tập con tương
0	000	$\emptyset$ (Tập rỗng
1	001	$\{A_0\}$
2	010	$\{A_1\}$
3	**011**	$\{A_0, A_1\}$
4	100	$\{A_2\}$
5	101	$\{A_0, A_2\}$
6	110	$\{A_1, A_2\}$
7	111	$\{A_0, A_1, A_2\}$

5. Kỹ thuật chia đôi tập – Meet in the Middle

Bài toán Subset Sum ($n \leq 40$)

- **Đề bài:** Cho mảng A gồm n phần tử ($n \leq 40$) và số nguyên S. Kiểm tra xem có tồn tại tập con nào của A có tổng bằng S hay không?
- **Nếu duyệt Bitmask trực tiếp:** Số tập con là $2^{40} \approx 10^{12}$ trạng thái. Vượt qua khả năng tính toán dẫn đến bị quá thời gian thực thi.

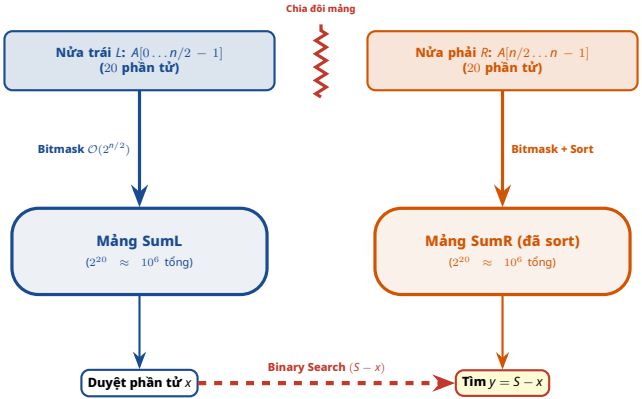
Ý tưởng Meet in the Middle

- Thay vì duyệt cả tập lớn 40 phần tử, ta **chia đôi** mảng thành 2 tập độc lập, mỗi tập chỉ gồm 20 phần tử.
- Khi đó, mỗi nửa chỉ có $2^{20} \approx 10^6$ tập con – hoàn toàn có thể vét cạn cực nhanh bằng Bitmask!
- Cuối cùng, ta ghép thông tin hai nửa lại bằng Tìm kiếm nhị phân.

Quy trình thực hiện thuật toán

1. **Bước 1 – Chia đôi (Divide):** Tách mảng A thành nửa trái $L = A[0 \dots n/2 - 1]$ và nửa phải $R = A[n/2 \dots n - 1]$.
2. **Bước 2 – Duyệt độc lập từng nửa (Brute Force):** Dùng Bitmask sinh toàn bộ tổng tập con của L lưu vào mảng SumL ($2^{n/2}$ phần tử) và toàn bộ tổng tập con của R lưu vào mảng SumR ($2^{n/2}$ phần tử).
3. **Bước 3 - Khảo sát và tính toán:** Sắp xếp tăng dần mảng SumR. Duyệt qua từng tổng $x \in$ SumL, dùng **Tìm kiếm nhị phân (Binary Search)** kiểm tra xem giá trị bù $y = S - x$ có xuất hiện trong SumR hay không.

Sơ đồ Kỹ thuật



- Thay vì tốn $2^{20} \times 2^{20} = 2^{40}$ phép toán để ghép mọi cặp (x, y), nhờ mảng SumR đã được sắp xếp, ta chỉ mất $\log_2(2^{20}) = 20$ bước tìm kiếm nhị phân cho mỗi phần tử $x \in$ SumL.

Cài đặt Mẫu: Bài toán Subset Sum với Meet in the Middle

1. Hàm sinh mọi tổng tập con bằng Bitmask

```cpp
vector<long long> gen_subsets(
    const vector<long long>& arr)
{
    int sz = arr.size();
    vector<long long> sums;
    // Duyet 2^(sz) trang thai
    for (int mask = 0; mask < (1 << sz); ++mask) {
        long long cur_sum = 0;
        for (int i = 0; i < sz; ++i) {
            if ((mask >> i) & 1) {
                cur_sum += arr[i];
            }
        }
        sums.push_back(cur_sum);
    }
    return sums;
}
```

2. Hàm chia đôi tập và Tìm kiếm nhị phân

```cpp
bool solve_subset_sum(
    int n, const vector<long long>& A, long long S)
{
    // Buoc 1: Chia doi mang A
    vector<long long> L(A.begin(), A.begin() + n/2);
    vector<long long> R(A.begin() + n/2, A.end());

    // Buoc 2: Sinh tong tap con cho 2 nua
    vector<long long> SumL = gen_subsets(L);
    vector<long long> SumR = gen_subsets(R);

    // Buoc 3: Sort SumR & Binary Search
    sort(SumR.begin(), SumR.end());
    for (long long x : SumL) {
        if (binary_search(SumR.begin(), SumR.end(), S - x
            ))
            return true; // Tim thay x + y = S
    }
    return false;
}
```

Thank You!

Meet in the Middle

Chia đôi mảng · Bitmask · Binary Search · $\mathcal{O}(n \cdot 2^{n/2})$

Hỏi & Đáp